MARS ROVER MANIPAL
RESEARCH AI TASKPHASE
Report 5

Adarsh Inaganti
240905294
adarshinaganti006@gmail.com

1. Logistic regression

1.1 Introduction to the dataset

The dataset tells us about a list of people who survived the crash of the Titanic. There are various columns indicating the passenger ID, class, name, sex, age, etc. The most important column is the `Survived` column which has a value of `0` or `1`, indicating whether the passenger survived the crash or not.

```
1  df = pd.read_csv('titanic.csv')
2  df.head()
```

✓ 0.0s Python

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

The `df` variable is the DataFrame which we get from the `titanic.csv` file.

```
1  # features and target
2
3  features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']
4  target = 'Survived'
```

✓ 0.0s

The `features` list tells us about the features our logistic regression model will use. The `target` is the `Survived` column as that is the data we want to classify.

```
# new df with only features and target

df_processed = df[features + [target]].copy()
df_processed.head()
```

✓ 0.0s

	Pclass	Sex	Age	SibSp	Parch	Fare	Survived
0	3	male	22.0	1	0	7.2500	0
1	1	female	38.0	1	0	71.2833	1
2	3	female	26.0	0	0	7.9250	1
3	1	female	35.0	1	0	53.1000	1
4	3	male	35.0	0	0	8.0500	0

We can make a new DataFrame called `df_processed` with only the required data.

1.2 Data cleaning and preparation

1.2.1 Handling missing values

```
# handle missing values

df_processed['Age'].fillna(df_processed['Age'].median(), inplace=True)
df_processed['Fare'].fillna(df_processed['Fare'].median(), inplace=True)
```

✓ 0.0s

We can fill the missing values in the Age and Fare columns with their median values. `inplace=True` makes sure that the changes are made directly.

1.2.2 Encoding categorical variables

```
# encode categorical variables

df_processed['Sex'] = df_processed['Sex'].map({'male': 0, 'female': 1})
```

✓ 0.0s

This converts the `Sex` column from text values (`male`, `female`) into numerical values (`0`, `1`). This is useful as it is easier to understand by our model.

1.2.3 Dropping NaN rows

```
# drop nan rows

df_processed.dropna(inplace=True)
```
✓ 0.0s

This drops null/missing values from our DataFrame.

1.2.4 Defining the feature matrix and target vector

In logistic regression, the dataset is divided into two parts.

1. **Feature matrix (X)**
    - It contains all the input features (predictors) used to make predictions.
    - Each row represents one data sample (observation).
    - Each column represents one feature (variable).
2. **Target vector (y)**
    - It contains the actual outcomes corresponding to each observation.
    - It's usually a column vector of shape (m, 1).
    - In logistic regression, y takes binary values — typically 0 or 1.

```
# X: feature matrix, y: target vector

X = df_processed[features].values
y = df_processed[target].values.reshape(-1, 1)
```
✓ 0.0s

- `X = df_processed[features].values`
    - `df_processed[features]`: This selects all the columns from the `df_processed` DataFrame whose names are stored in a variable called `features`.
    - `.values`: This attribute converts the resulting pandas DataFrame into a NumPy array.

- `y = df_processed[target].values.reshape(-1, 1)`
  - `df_processed[target]`: This selects the single column from `df_processed` whose name is stored in the `target` variable (e.g., target = 'Survived').
  - `.values`: This converts the pandas Series (a single column) into a 1-dimensional NumPy array.
  - `.reshape(-1, 1)`: This is a crucial step. It changes the shape of the 1D array.
    - An array like `[0, 1, 1, 0]` (shape: `(4,)`) becomes `[[0], [1], [1], [0]]` (shape: `(4, 1)`).
    - The `-1` tells NumPy to automatically figure out the number of rows.
    - The `1` explicitly states that there should be one column.

1.3 Training the regression model

1.3.1 Manual test-train split

```
1  # manually split data (80/20)
2
3  np.random.seed(69)
4  m = X.shape[0] # total number of samples
5  indices = np.random.permutation(m)
✓  0.0s
```

- `np.random.seed(69)` sets a seed for NumPy's random number generator. This is essential for reproducibility.
- The variable `m` is used to store the total number of samples.
- The variable `indices` stores an array containing every integer from `0` to `m-1` in a randomly shuffled order.

```
# shuffle X and y using same indices

X_shuffled = X[indices]
y_shuffled = y[indices]
```
✓ 0.0s

`X_shuffled` and `y_shuffled` are new arrays which store rows from X and y respectively in a specific, random order defined by `indices`.

```
# split index

split_idx = int(0.8 * m)
```
✓ 0.0s

The `split_idx` defines the split index. This means that the data is split so that 80% of the data is training data and 20% is test data. `int(0.8 * m)` is used to typecast the result to an integer.

1.3.2 Slicing

```
# slicing

X_train_raw = X_shuffled[:split_idx]
y_train = y_shuffled[:split_idx]
X_test_raw = X_shuffled[split_idx:]
y_test = y_shuffled[split_idx:]
```

✓ 0.0s

- `X_train_raw` and `X_test_raw` are the raw training and raw test feature set respectively.
    - It is made by choosing the first 80% and last 20% of the rows of `X_shuffled` respectively.
    - This is called slicing.
    - It is raw as this data is not standardized and scaled yet.
- This same slicing operation is done on `y_train` and `y_test`.

1.3.3 Feature scaling

```
# feature scaling

train_mean = np.mean(X_train_raw, axis=0)
train_std = np.std(X_train_raw, axis=0) + 1e-8 # avoid division by zero

X_train_scaled = (X_train_raw - train_mean) / train_std
X_test_scaled = (X_test_raw - train_mean) / train_std
```

✓ 0.0s

- `train_mean` and `train_std` denote the mean and standard deviation of `X_train_raw` respectively.
    - The `1e-8` is added to `train_std` to prevent a division error if a feature has zero variance.
- `X_train_scaled` and `X_test_scaled` are the scaled training and test data.

1.3.4 Adding a bias term

```
# adding bias term

intercept_train = np.ones((X_train_scaled.shape[0], 1))
X_train = np.hstack((intercept_train, X_train_scaled))

intercept_test = np.ones((X_test_scaled.shape[0], 1))
X_test = np.hstack((intercept_test, X_test_scaled))
```

✓ 0.0s

- `intercept_train` and `intercept_test` creates a new array filled only with ones for the training and test data respectively.
    - The shape is `(number of test/train samples, 1)`.
    - It creates a column vector full of ones with the same height as the training/test data.
- `X_train` and `X_test` creates a horizontal stack (stacks columns side by side) for the training and test data respectively.
    - It stacks `intercept_train`/`intercept_test` (the column vector full of ones) with the scaled data `X_train_scaled`/`X_test_scaled`.

1.3.5 Defining the sigmoid and cost function

```
# sigmoid function

def sigmoid(z):
    z_clipped = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z_clipped))

def compute_cost(X, y, weights):
    m = len(y)
    z = np.dot(X, weights)
    h = sigmoid(z)
    epsilon = 1e-8
    cost = (-1 / m) * np.sum(y * np.log(h + epsilon) + (1 - y) * np.log(1 - h + epsilon))
    return cost
```

✓ 0.0s

1. **`sigmoid()`**
    - The function takes one argument: `z`.
    - `z_clipped` ensures that:
        - If `z` > 500, set `z` to 500.
        - If `z` < -500, set `z` to -500.

        - This is done to prevent overflow or underflow of the `np.exp()` function.
    - It returns the formula of the sigmoid function on `z_clipped`, as it is safer to use than `z`.
    - The sigmoid function is defined as:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

2. **`compute_cost()`**
    - It takes three arguments:
        - `X`: feature matrix
        - `y`: target vector
        - `weights`: weights of the current model
    - The variable `m` takes the length of `y`, getting the number of training samples.
    - The variable `z` is a dot product of the feature matrix (`X`, which includes the bias term) and the current `weights` of the model.
    - The number `epsilon` is a very small number (`1e-8`). This is to prevent `np.log(0)`, which would result in `-inf` (negative infinity).
    - Finally, the `cost` variable stores the result of the cost function and returns it.
    - The cost function is defined as:

$$Cost(h, y) = -\frac{1}{m}\left[y^{(i)} log(h^{(i)}) + (1 - y^{(i)}) log(1 - h^{(i)})\right]$$

1.3.6 Defining the gradient descent function

```python
# gradient descent function

def gradient_descent(X, y, weights, alpha, iterations):
    m = len(y)
    cost_history = []
    iteration_points = []

    for i in range(iterations):
        z = np.dot(X, weights)
        h = sigmoid(z)
        gradient = (1 / m) * np.dot(X.T, (h - y))
        weights -= alpha * gradient
        cost = compute_cost(X, y, weights)
        cost_history.append(cost)
        iteration_points.append(i)

    return weights, cost_history, iteration_points
```

✓ 0.0s

- It takes five arguments:
  - `X`: feature matrix
  - `y`: target vector
  - `weights`: the initial guess for the weights
  - `alpha`: learning rate
  - `iterations`: total number of iterations
- The variable `m` stores the total number of training samples.
- The list `cost_history` stores the cost value from each iteration. Initially, it is an empty list.
- The list `iteration_points` is another empty list which stores the iteration number to plot the cost vs iterations graph later.
- ***Gradient descent loop***
  - The variable `z` is a dot product of the feature matrix (`X`) and the current `weights`. It denotes the linear predictions.
  - The variable `h` is the model's hypothesis. It is calculated by passing the linear predictions (`z`) through the `sigmoid` function.
  - The `gradient` variable stores the current gradient by calculating it using the formula:

$$Gradient = \frac{1}{m}(X^T \cdot (h - y))$$

  - The `weights` are updated by subtracting them from the product of the learning rate (`alpha`) and the `gradient`.
  - The new `cost` is calculated by using the `compute_cost` function and by passing the updated `weights`.

    - The new calculated `cost` is appended to the `cost_history` list.
    - The current iteration `i` is appended to the `iteration_points` list.
- The function returns:
    - The final trained `weights`.
    - The `cost_history` list.
    - The `iteration_points` list.

1.3.7 Performing gradient descent

```
# train the model

alpha = 0.05
iterations = 10000

n_features = X_train.shape[1]
weights_initial = np.zeros((n_features, 1))
```

✓ 0.0s

- The hyperparameters of learning rate (`alpha`) and `iterations` are set to 0.05 and 10,000 respectively.
- The variable `n_features` stores the number of features.
- The variable `weights_initial` is a zero matrix, as all the weights are zero initially.

```
# run gradient descent
trained_weights, cost_history, iteration_points = gradient_descent(X_train, y_train, weights_initial, alpha, iterations)
```

✓ 2.1s Python

The gradient descent is performed by passing the parameters:

- Feature matrix (`X_train`)
- Target vector (`y_train`)
- Initial weights (`weights_initial`)
- Learning rate (`alpha`)
- Number of iterations (`iterations`).

1.3.8 Defining the functions to predict and calculate accuracy

```python
# prediction function

def predict(X, weights, threshold=0.5):
    z = np.dot(X, weights)
    probabilities = sigmoid(z)
    predictions = (probabilities >= threshold).astype(int)
    return predictions

def calculate_accuracy(y_true, y_pred):
    correct_predictions = (y_true == y_pred)
    accuracy = np.mean(correct_predictions)
    return accuracy
```

✓ 0.0s

1. **`predict()`**
    - It takes three arguments:
        - `X`: feature matrix
        - `weights`: trained weights
        - `threshold` (optional): defaults to `0.5`
    - The variable `z` is the dot product of the feature matrix (`X`) and the trained `weights`.
    - The variable `probabilities` is the result of the `sigmoid` function on `z`.
    - The `predictions` array performs an element-wise comparison of `probabilities` with the `threshold`.
        - If `probabilities` is greater than or equal to `0.5`, the result is `True`.
        - Otherwise, it is `False`.
    - `.astype(int)` converts the `True` and `False` values to `1` and `0` respectively.
    - The function returns the final `predictions` array.
2. **`calculate_accuracy()`**
    - It takes two arguments:
        - `y_true`: the ground-truth labels
        - `y_pred`: predictions of the model from the `predict` function
    - The variable `correct_predictions` is a boolean array which checks if the true labels (`y_true`) and the predictions (`y_pred`) are equal element-wise.

- The `accuracy` variable calculates the accuracy of the boolean array.
  - It is calculated by finding the `mean` of the array (number of true predictions divided by the total number of predictions).
  - This is easy to calculate mathematically as NumPy treats `True` as `1` and `False` as `0`.
- The function returns the final `accuracy`.

1.3.9 Calculating predictions and accuracy

```
1  y_pred = predict(X_test, trained_weights)
2  accuracy = calculate_accuracy(y_test, y_pred)
3  print(f"Model Accuracy on Test Set: {accuracy * 100:.2f}%")
✓  0.0s

Model Accuracy on Test Set: 80.45%
```

- The `y_pred` variable stores the array of predictions returned by the predict function on the training data (`X_test`) and the trained weights (`trained_weights`).
- The `accuracy` variable stores the accuracy of the model returned by the calculate_accuracy function on the testing data (`y_test`) and the predicted data (`y_pred`).
- From the calculations, we can see that our model's accuracy is at 80.45%.

1.3.10 Calculating the final model weights

```
1  # final model weights
2
3  all_features = ['Intercept'] + features
4  weights_series = pd.Series(trained_weights.flatten(), index=all_features)
5  weights_series.sort_values(ascending=False)
✓  0.0s
```

- The `all_features` list contains the list of all the `features` (which we made earlier), and the `Intercept` column.
- `weights_series` is a pandas Series which contains all the trained weights (trained_weights).
  - `.flatten()` is used to convert a 2D array to a 1D array.
  - `index=all_features` treats the `all_features` variable as the `index` for our Series. This makes sure that all the features are lined up correctly with their respective trained weights.

- The final line is used to print the `weights_series` in descending order.

```
Sex          1.315686
Fare         0.076541
Parch       -0.004208
SibSp       -0.367884
Age         -0.464642
Intercept   -0.689986
Pclass      -0.994678
dtype: float64
```

These are the final trained weights for our model.

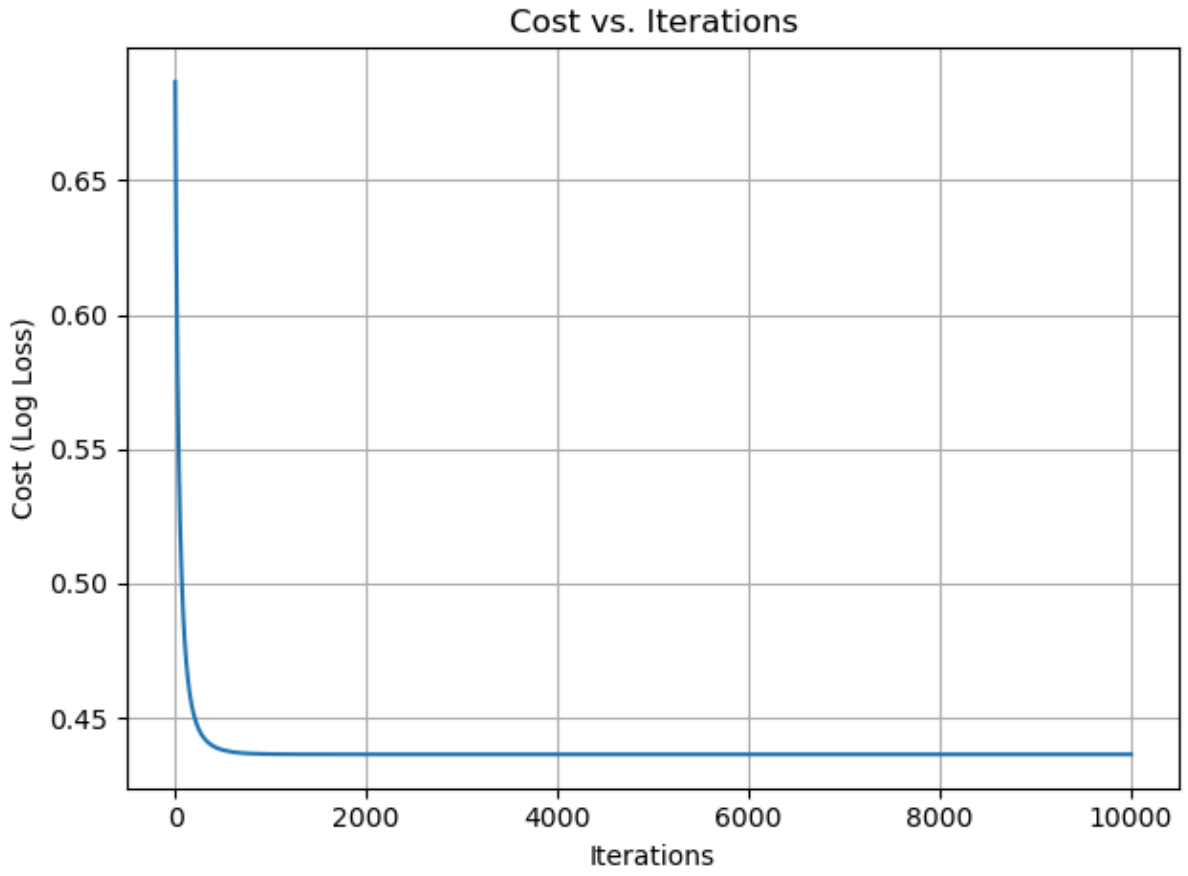
1.3.11 Plotting the cost vs iterations graph

The cost vs iterations graph (also called the learning curve) helps us to determine a proper learning rate (`alpha`) for our model. It can also be used to detect overfitting or underfitting.

```
1  # cost vs iterations plot
2
3  plt.plot(iteration_points, cost_history)
4  plt.xlabel("Iterations")
5  plt.ylabel("Cost (Log Loss)")
6  plt.title("Cost vs. Iterations")
7  plt.grid(True)
8  plt.tight_layout()
✓  0.0s
```

The `plt` keyword is used for the `matplotlib` library which is used to plot the graph.

- The code plots a graph.
    - The x-axis is taken as `iteration_points`, which is the list containing all the iterations.
    - The y-axis is taken as `cost_history`, the list which contains all the costs from our gradient descent function.
- The `xlabel`, `ylabel`, `title` and `grid` are added to improve readability.
- `plt.tight_layout()` automatically adjusts the plot's padding and spacing to ensure all elements fit neatly without overlapping.



The graph tells us that the hyperparameters of learning rate (alpha) and the number of iterations were chosen correctly.

1. ***Rapid initial learning:***
    - The graph starts at a high cost (around 0.68), which is the error from the initial guess weights (all zeros).
    - The cost then drops steeply by around 1500 iterations, which tells us that the gradient descent model is efficient in reducing the cost.
2. ***Convergence:***
    - After the initial drop, the curve flattens out and the cost decreases much more slowly. This is known as convergence.

- The model is approaching the lowest possible cost and the updates to the weights are becoming very small.